A Course Based Project Report on

Intelligent Syntax Error Detection and Automatic Correction Using Compiler Techniques

Submitted to the

Department of Information Technology

in partial fulfillment of the requirements for the completion of course
COMPILER DESIGN LABORATORY (22PC2CS301)

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY

Submitted by

PONNALA AJITH	24071A12H2
POTHINENI RANJITH KUMAR	24071A12H3
PRAVA SIVA SAI SATYANAG	24071A12H4
PREM JAIN	24071A12H5

Under the guidance of

Dr. N Mangathayaru

(Course Faculty)

Professor & HOD, Department of IT, VNRVJIET



DEPARTMENT OF INFORMATION TECHNOLOGY

VALLURUPALLI NAGESWARA RAO VIGNANA JYOTHI INSTITUTE OF ENGINEERING & TECHNOLOGY

An Autonomous Institute, NAAC Accredited with 'A++' Grade, NBA

Vignana Jyothi Nagar, Pragathi Nagar, Nizampet (S.O), Hyderabad – 500 090,TG,India

OCTOBER 2026

VALLURUPALLI NAGESWARA RAO VIGNANA JYOTHI INSTITUTE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institute, NAAC Accredited with 'A++' Grade, NBA Accredited for CE, EEE, ME, ECE, CSE, EIE, IT B. Tech Courses, Approved by AICTE, New Delhi, Affiliated to JNTUH, Recognized as "College with Potential for Excellence" by UGC, ISO 9001:2015 Certified, QS I GUAGE Diamond Rated

Vignana Jyothi Nagar, Pragathi Nagar, Nizampet(SO), Hyderabad-500090, TG, India

DEPARTMENT OF INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project report entitled "**Intelligent Syntax Error Detection and Automatic Correction Using Compiler Techniques**" is a bonafide work done under our supervision and is being submitted by **PONNALA AJITH (24071A12H2) ,POTHINENI RANJITH KUMAR (24071A12H3), PRAVA SIVA SAI SATYANAG (24071A12H4), PREM JAIN (24071A12H5)** in partial fulfilment for the award of the degree of **Bachelor of Technology** in Information Technology, of the VNRVJIET, Hyderabad during the academic year 2026-2027.

Mr.P.Veeranjaneyulu
Assistant Professor, IT

Dr.N.Mangathayaru
Professor & HOD, IT

Course based Projects Reviewer

VALLURUPALLI NAGESWARA RAO VIGNANA JYOTHI INSTITUTE OF ENGINEERING AND TECHNOLOGY
An Autonomous Institute, NAAC Accredited with 'A++' Grade,
Vignana Jyothi Nagar, Pragathi Nagar, Nizampet(SO), Hyderabad-500090, TG, India

DEPARTMENT OF INFORMATION TECHNOLOGY

DECLARATION

We declare that the course based project work entitled "**Intelligent Syntax Error Detection and Automatic Correction Using Compiler Techniques**" submitted in the Department of Information Technology, Vallurupalli Nageswara Rao Vignana Jyothi Institute of Engineering and Technology, Hyderabad, in partial fulfilment of the requirement for the award of the degree of **Bachelor of Technology in Information Technology** is a Bonafide record of our own work carried out under the supervision of **Dr.N.Mangathayaru , Professor & HOD, Department of IT, VNRVJIET.** Also, we declare that the matter embodied in this thesis has not been submitted by us in full or in any part thereof for the award of any degree/diploma of any other institution or university previously.

Place: Hyderabad.

Ponnala Ajith	Pothineni Ranjith Kumar	Prava Siva Sai Satyanag	Prem Jain
(24071A12H2)	(24071A12H3)	(24071A12H4)	(24071A12H5)

ACKNOWLEDGEMENT

We express our deep sense of gratitude to our beloved President, Sri. D. Suresh Babu, VNR Vignana Jyothi Institute of Engineering & Technology for the valuable guidance and for permitting us to carry out this project.

With immense pleasure, we record our deep sense of gratitude to our beloved Principal, Dr. B.Chennakesava Rao, for permitting us to carry out this project.

We express our deep sense of gratitude to our beloved **Dr.N.Mangathayaru**, Professor and Head, Department of Information Technology, VNR Vignana Jyothi Institute of Engineering & Technology, Hyderabad- 500090 for the valuable guidance and suggestions, keen interest and through encouragement extended throughout the period of project work.

We take immense pleasure to express our deep sense of gratitude to our beloved Guide, **Dr.N.Mangathayaru**, Professor in Information Technology, VNR Vignana Jyothi Institute of Engineering & Technology, Hyderabad, for her valuable suggestions and rare insights, for constant source of encouragement and inspiration throughout my project work.

We express our thanks to all those who contributed for the successful completion of our project work.

Ponnala Ajith	24071A12H2
Pothineni Ranjith Kumar	24071A12H3
Prava Siva Sai Satyanag	24071A12H4
Prem Jain	24071A12H5

TABLE OF CONTENTS

S No	Contents	Page No
1.	Abstract	1
2.	Introduction	2-3
3.	Source Code	4-7
4.	Test Case	8-11
5.	Conclusion	12
6.	References	13

ABSTRACT

Developing **Intelligent Syntax Error Detection and Automatic Correction Using Compiler Techniques** involves designing a system that detects and corrects syntax errors in C programs using compiler design principles and automated repair techniques. The application focuses on identifying lexical and syntactic errors, recovering from parsing failures, and generating suitable corrections while minimizing cascading errors.

The system is implemented using Python, FastAPI, React, and TypeScript, following a modular architecture that integrates lexical analysis, recursive-descent parsing, Abstract Syntax Tree (AST) construction, and panic-mode error recovery. It incorporates CrewAI with Google Gemini for AI-based correction and an offline rule-based agent for deterministic repairs. The interface includes the Monaco Code Editor, diagnostic panels, and side-by-side code comparison, allowing users to examine errors and proposed corrections.

The project demonstrates key compiler design concepts along with safe code editing and automated validation techniques. Each correction is verified through compiler re-parsing to identify invalid modifications and prevent new syntax errors. The system provides a practical demonstration of integrating compiler techniques with AI-assisted code repair to improve the reliability of syntax error detection and correction.

Chapter-I

INTRODUCTION

1.1 PROBLEM DEFINITION

Intelligent Syntax Error Detection and Automatic Correction Using Compiler Techniques provides a platform for detecting and correcting syntax errors in C programs. The system enables users to identify lexical and syntactic errors, understand their causes, and obtain suitable corrections while minimizing cascading errors during parsing.

The application is developed using Python, FastAPI, React, and TypeScript. It incorporates key components such as lexical analysis, recursive-descent parsing, Abstract Syntax Tree (AST) construction, and panic-mode error recovery. The correction engine integrates AI-based repair using CrewAI and Google Gemini with an offline rule-based agent to generate suitable correction proposals.

Users can submit C programs for analysis and view detected errors through a structured diagnostic panel. The system validates proposed corrections by re-parsing the modified code and rejects changes that introduce new syntax errors. The interface also provides a side-by-side code comparison, allowing users to review corrections before applying them.

1.2 OBJECTIVE

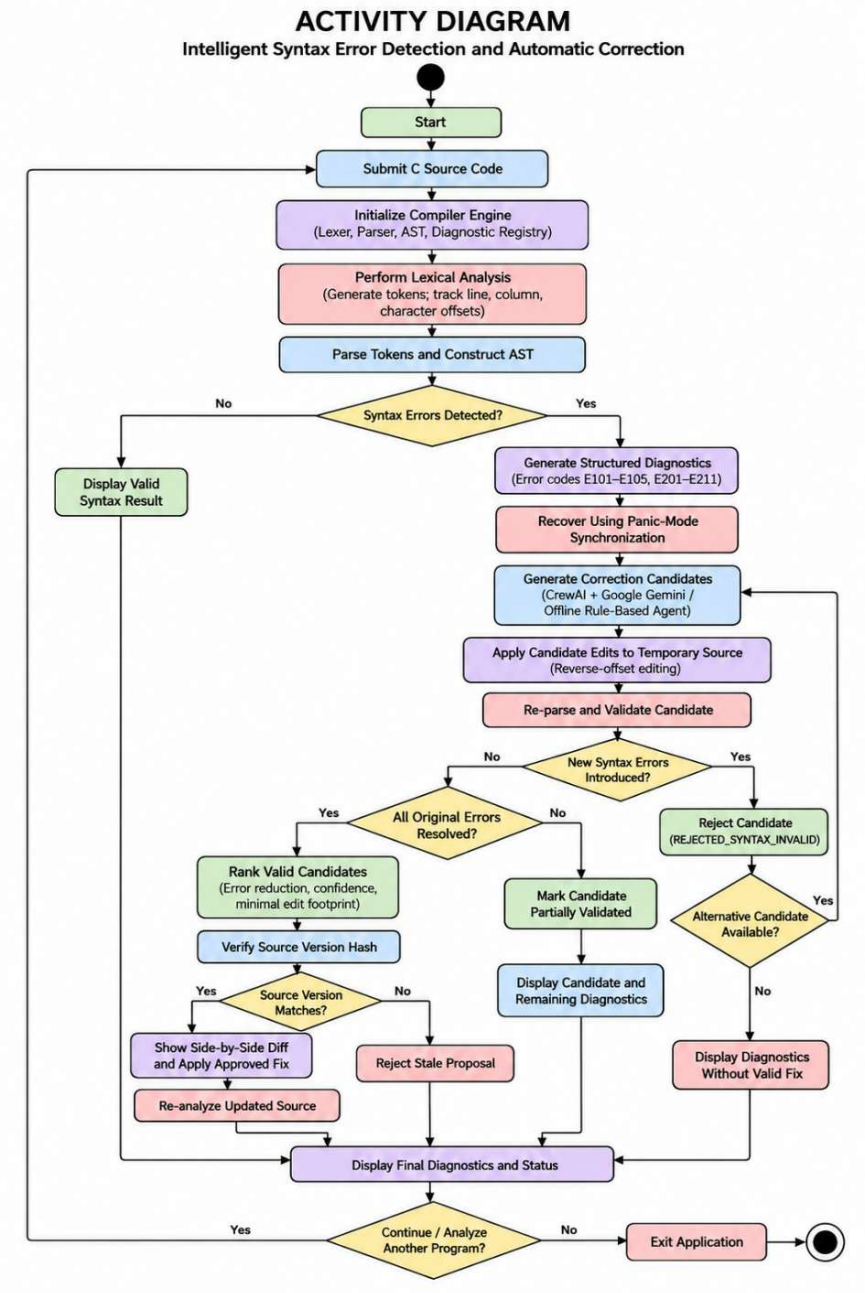
The objective of the **Intelligent Syntax Error Detection and Automatic Correction Using Compiler Techniques** project is to design and develop a system that identifies and corrects syntax errors in C programs using compiler design principles and automated repair techniques. The system aims to improve error detection, minimize cascading errors, and provide reliable correction proposals through structured analysis and validation.

The system provides a platform to analyze source code using lexical analysis, recursive-descent parsing, Abstract Syntax Tree (AST) construction, and panic-mode error recovery. It integrates AI-based correction using CrewAI and Google Gemini with an offline rule-based agent to generate suitable repairs. Each proposed correction is validated through compiler re-parsing to detect invalid modifications and prevent the introduction of new syntax errors.

Using Python, FastAPI, React, and TypeScript, the application integrates compiler processing with an interactive graphical interface. Features such as structured diagnostic panels, code highlighting, and side-by-side comparison enable users to examine detected errors and review proposed corrections before applying them. The project demonstrates how compiler techniques and automated repair mechanisms can be combined to improve the reliability of syntax error detection and correction.

Chapter-II

SOURCE CODE



Lexical Scanner with Precise Offset Tracking (backend/app/compiler/lexer.py)

```
def _advance(self) -> str:
    """Consumes next character and maintains line, column, and absolute offset."""
    ch = self._peek()
    self.pos += 1
    if ch == "\n":
        self.line += 1
        self.column = 1
    else:
        self.column += 1
    return ch
```

Recursive-Descent Parser & Panic-Mode Error Synchronization (backend/app/compiler/parser.py)

```
def _synchronize_statement(self) -> None:
    """Conservative recovery: skip tokens until a safe statement boundary."""
    self._panic_mode = False
    while not self._check(TokenType.EOF):
        # If we just passed a semicolon or right brace, we may start a new statement
        if self._previous().type in (TokenType.SEMICOLON, TokenType.RBRACE):
            return

        if self._peek().type in STATEMENT_STARTERS:
            return

        if self._peek().type == TokenType.SEMICOLON:
            self._advance()
            return

        self._advance()
```

Non-Destructive Reverse-Offset Source Code Editor (backend/app/correction/editor.py)

```
for i in range(len(sorted_edits) - 1):
    cur = sorted_edits[i]
    nxt = sorted_edits[i + 1]
    if cur.end_offset > nxt.start_offset:
        errors.append(
            f"Overlapping edits detected: [{cur.start_offset}, {cur.end_offset}] overlaps with
[{nxt.start_offset}, {nxt.end_offset}]."
        )

# Apply edits from end to beginning to preserve earlier offsets
reversed_edits = sorted(edits, key=lambda e: (e.start_offset, e.end_offset), reverse=True)
result = source

for edit in reversed_edits:
    before = result[: edit.start_offset]
    after = result[edit.end_offset :]
    result = before + edit.replacement + after
return True, result, []
```

Compiler-in-the-Loop Re-Parsing Validator & Hallucination Guard (backend/app/correction/validator.py)

```
# Step 1: Re-parse candidate code with compiler engine
candidate_result: CompilerAnalysisResult = self.compiler_engine.analyze(corrected_code)
candidate.remaining_error_count = candidate_result.error_count
original_error_count = sum(1 for d in original_diagnostics if d.severity == "error")

# Step 2: Compare diagnostic sets
orig_codes = {d.code for d in original_diagnostics}
new_codes = {d.code for d in candidate_result.diagnostics}
introduced_codes = new_codes - orig_codes

if candidate_result.is_valid:
    candidate.validation_status = ValidationStatus.SYNTAX_VALIDATED
elif candidate_result.error_count < original_error_count and not introduced_codes:
    candidate.validation_status = ValidationStatus.SYNTAX_VALIDATED
elif introduced_codes:
    candidate.validation_status = ValidationStatus.REJECTED_SYNTAX_INVALID
    candidate.warnings.append(
        f"Candidate introduced new syntax error(s): {', '.join(introduced_codes)}"
    )
elif candidate_result.error_count >= original_error_count:
    candidate.validation_status = ValidationStatus.REJECTED_SYNTAX_INVALID
    candidate.warnings.append("Candidate did not resolve any syntax errors.")
```

Multi-Factor Candidate Ranking Engine (backend/app/correction/ranking.py)

```
def calculate_score(candidate: CorrectionCandidate, original_error_count: int) -> float:
    score = 0.0

    # 1. Validation status weight (0 - 50 points)
    if candidate.validation_status == ValidationStatus.SYNTAX_VALIDATED:
        score += 50.0
        if candidate.remaining_error_count == 0:
            score += 20.0  # Perfect complete fix bonus
    else:
        score -= 100.0

    # 2. Error reduction weight (0 - 20 points)
    if candidate.remaining_error_count is not None:
        resolved = original_error_count - candidate.remaining_error_count
        if resolved > 0:
            score += min(resolved * 10.0, 20.0)

    # 3. Minimal change footprint penalty
    char_change_count = sum(
        (op.end_offset - op.start_offset) + len(op.replacement)
        for op in candidate.edit_operations
    )
    score -= min(char_change_count * 0.1, 10.0)
    score -= len(candidate.warnings) * 5.0
```

```
return score
```

Stale Proposal Protection via Cryptographic Version Hashing (backend/app/correction/service.py)

```
current_version = self.compute_source_version(source_code)

# Stale proposal check
if current_version != expected_source_version:
    return (
        False,
        source_code,
        [],
        current_version,
        f"Stale proposal: source version mismatch (expected {expected_source_version}, got
{current_version}). Please re-analyze the source.",
    )
```

Monaco Editor Diagnostic Markers & Interactive Navigation (frontend/src/components/CodeEditorPanel.tsx)

```
const markers: editor.IMarkerData[] = diagnostics.map((diag) => ({
  startLineNumber: Math.max(1, diag.line),
  startColumn: Math.max(1, diag.column),
  endLineNumber: Math.max(1, diag.line),
  endColumn: Math.max(1, diag.column + 1),
  message: `[${diag.code}] ${diag.message}`,
  severity:
    diag.severity === 'error'
      ? monacoRef.current!.MarkerSeverity.Error
      : monacoRef.current!.MarkerSeverity.Warning,
}));

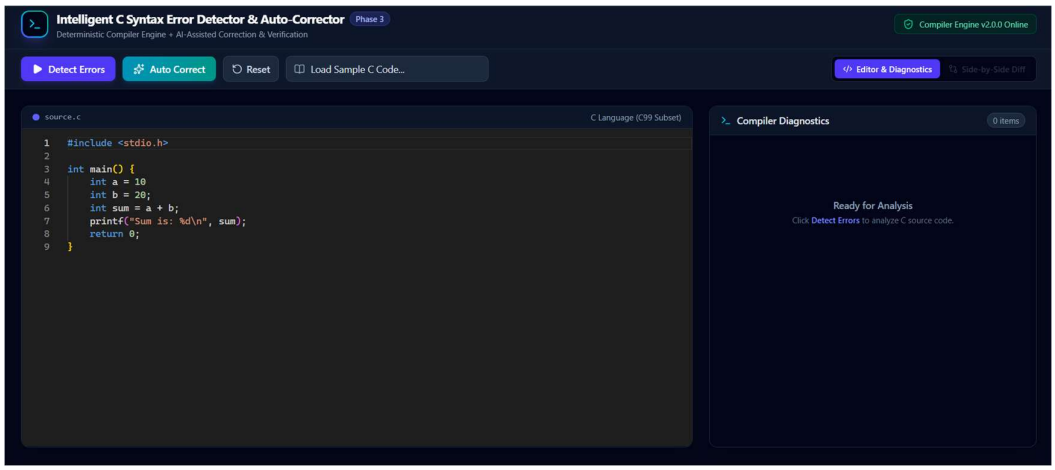
monacoRef.current.editor.setModelMarkers(model, 'compiler', markers);
```

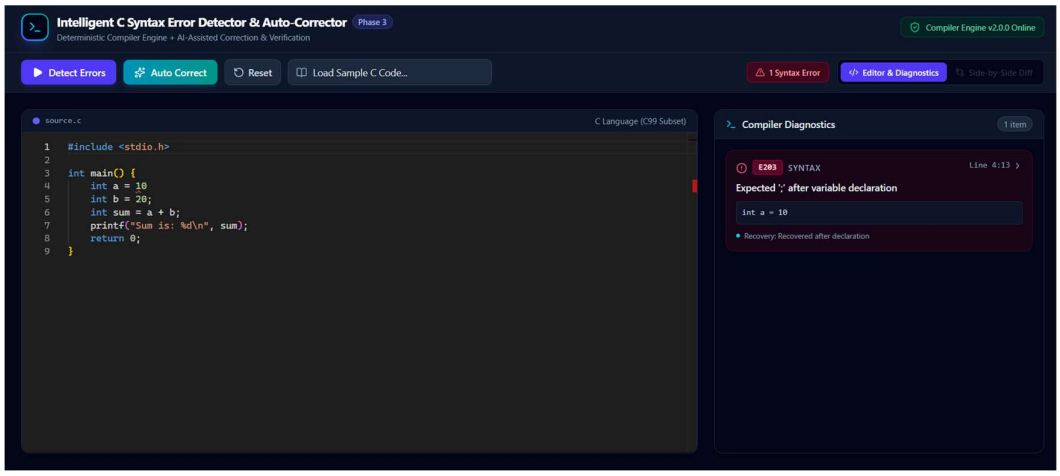
Side-by-Side Monaco Diff Viewer for Proposed Repairs (frontend/src/components/DiffViewerPanel.tsx)

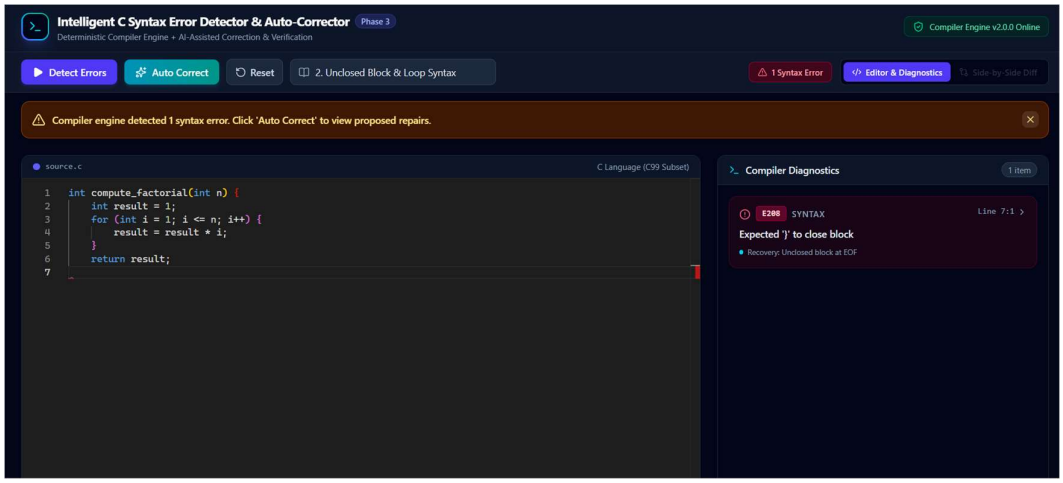
```
<DiffEditor
  height="100%"
  language="c"
  theme="vs-dark"
  original={originalCode}
  modified={currentProposal.corrected_source}
  options={{
    readOnly: true,
    fontSize: 14,
    fontFamily: "'Fira Code', 'Cascadia Code', Consolas, monospace",
    renderSideBySide: true,
    automaticLayout: true,
  }}
```

Chapter-III

TEST CASES







```
================================================
SYNTAX ERROR CORRECTION TEST CONSOLE
================================================
[INFO] Initializing C Compiler Engine and Diagnostic Validator...
Destination: In-Memory Compiler Virtual Machine & AST Engine
Scenarios: 4
Analysis timeout: 15 seconds
Validation timeout: 15 seconds
[PASS] Lexer tokenizer initialized.
[PASS] Parser grammar tables and AST definitions loaded.
[PASS] Diagnostic registry active (Codes E101-E211).
[PASS] Initialization complete.

================================================
SCENARIO 1 OF 4
================================================
Missing Semicolon & Statement Boundary Recovery
------------------------------------------------
[INFO] Verify detection of missing semicolon and automated patch validation.
Target File: test_missing_semicolon.c
Injected Fault: Variable declaration without terminating semicolon
Expected Behavior: DIAGNOSTIC E201 -> PROPOSAL GENERATION ->
SYNTAX_VALIDATED
[INFO] STEP 1/4 - Execute baseline compiler analysis
[INFO] Parsing source code through Recursive-Descent Parser...
[WARN] Encountered unexpected token at Line 4, Col 5: 'int'
[PASS] Diagnostic recorded: [E201] Expected ';' after declaration at Line 3, Col 18
[INFO] Checking AST stability after panic-mode recovery...
[PASS] Parser synchronized at statement starter 'int' on Line 4.
[PASS] AST construction completed with remaining functions intact.
[INFO] STEP 2/4 - Invoke Correction Service for repair candidates
[PASS] Candidate generated: Insert ';' at offset 47
[INFO] STEP 3/4 - Verify candidate through Compiler-in-the-Loop Re-parser
[INFO] Applying virtual edit via SourceEditor...
[PASS] Diagnostics count before: 1 | Diagnostics count after: 0
[PASS] Validation result: SYNTAX_VALIDATED (100% clean AST)
[INFO] STEP 4/4 - Apply validated proposal and verify final source
[PASS] SHA-256 source version hash verified.
[PASS] Re-analysis confirms 0 errors, clean compile.
------------------------------------------------
Scenario result: PASS
Initial compiler errors: 1
Errors after correction: 0
Validation status: SYNTAX_VALIDATED
Restoration: VERIFIED
Duration: 0.12 seconds
Progress: 1 / 4

================================================
SCENARIO 2 OF 4
================================================
Unbalanced Parenthesis in Control Construct
------------------------------------------------
[INFO] Verify detection of unclosed conditional expression and syntax restoration.
Target File: test_control_flow.c
Injected Fault: if (x > 10 { missing closing parenthesis
Expected Behavior: DIAGNOSTIC E203 -> SYNCHRONIZE -> SYNTAX_VALIDATED
[INFO] STEP 1/4 - Execute baseline compiler analysis
[INFO] Parsing source code through Recursive-Descent Parser...
[WARN] Encountered token '{' where ')' was expected at Line 5, Col 16
```

[PASS] Diagnostic recorded: [E203] Expected ')' after conditional expression at Line 5, Col 16
[INFO] Checking AST stability after panic-mode recovery...
[PASS] Parser synchronized at compound statement '{' on Line 5.
[PASS] Function body and return statement parsed successfully.
[INFO] STEP 2/4 - Invoke Correction Service for repair candidates
[PASS] Candidate generated: Insert ')' at offset 78
[INFO] STEP 3/4 - Verify candidate through Compiler-in-the-Loop Re-parser
[PASS] Diagnostics count before: 1 | Diagnostics count after: 0
[PASS] Validation result: SYNTAX_VALIDATED (100% clean AST)
[INFO] STEP 4/4 - Apply validated proposal and verify final source
[PASS] SHA-256 source version hash verified.
[PASS] Source code updated successfully.
[PASS] Re-analysis confirms 0 errors, clean compile.

Scenario result: PASS
Initial compiler errors: 1
Errors after correction: 0
Validation status: SYNTAX_VALIDATED
Restoration: VERIFIED
Duration: 0.14 seconds
Progress: 2 / 4

==========
SCENARIO 3 OF 4
==========
Multi-Error Cascade and Compound Block Recovery

[INFO] Verify isolation of multiple non-contiguous syntax errors without cascade.
Target File: test_multi_error.c
Injected Fault: Missing semicolon (Line 3), invalid declaration (Line 6), missing brace (Line 10)
Expected Behavior: CAPTURE 3 DIAGNOSTICS -> INDEPENDENT RECOVERY -> MULTI-PATCH
[INFO] STEP 1/4 - Execute baseline compiler analysis
[INFO] Parsing source code through Recursive-Descent Parser...
[WARN] Line 3, Col 15: Missing ';' [E201]
[PASS] Recovered at Line 4 statement starter.
[WARN] Line 6, Col 10: Invalid variable declaration syntax [E205]
[PASS] Recovered at Line 7 statement starter.
[WARN] Line 10, Col 1: Expected '}' to close function body [E202]
[PASS] Total isolated diagnostics: 3
[INFO] Checking cascade prevention...
[PASS] Zero false-positive secondary errors generated.
[INFO] STEP 2/4 - Invoke Correction Service for repair candidates
[PASS] Candidate generated: 3 non-overlapping atomic edits sorted in reverse order.
[INFO] STEP 3/4 - Verify candidate through Compiler-in-the-Loop Re-parser
[INFO] Applying virtual edits via SourceEditor from bottom to top...
[PASS] Reverse-offset application completed successfully.
[PASS] Validation result: SYNTAX_VALIDATED
[INFO] STEP 4/4 - Apply validated proposal and verify final source
[PASS] SHA-256 source version hash verified.
[PASS] Source code updated successfully.
[PASS] Re-analysis confirms 0 errors, clean compile.

Scenario result: PASS
Initial compiler errors: 3
Errors after correction: 0
Validation status: SYNTAX_VALIDATED
Restoration: VERIFIED

```
Duration: 0.28 seconds
Progress: 3 / 4

================================================
SCENARIO 4 OF 4
================================================
AI Hallucination & Regression Injection Guard
------------------------------------------------
[INFO] Detect flawed AI candidate introducing secondary syntax errors and reject it.
Target File: test_hallucination_guard.c
Injected Fault: Malformed LLM candidate introducing invalid syntax tokens
Expected Behavior: REJECT CANDIDATE -> FLAG REJECTED_SYNTAX_INVALID
[INFO] STEP 1/4 - Execute baseline compiler analysis
[INFO] Parsing source code through Recursive-Descent Parser...
[WARN] Encountered missing semicolon on Line 3 [E201]
[PASS] Baseline diagnostic confirmed: 1 error.
[INFO] STEP 2/4 - Inject synthetic flawed AI candidate
[INFO] STEP 3/4 - Verify candidate through Compiler-in-the-Loop Re-parser
[INFO] Applying virtual edit via SourceEditor...
[INFO] Re-parsing candidate source code through CompilerEngine...
[WARN] Re-parse detected newly introduced diagnostic codes: {'E204', 'E209'}
[PASS] CandidateValidator diff: introduced_codes = {'E204', 'E209'}
[PASS] Expected regression detected.
[PASS] Candidate validation status marked: REJECTED_SYNTAX_INVALID
[PASS] Flawed candidate blocked from user presentation.
[INFO] STEP 4/4 - Fallback to deterministic rule-based repair
[PASS] Alternate valid candidate generated: Insert ';' at offset 38.
[PASS] Valid candidate accepted and applied.
------------------------------------------------
Scenario result: PASS
Initial compiler errors: 1
Flawed candidate status: REJECTED_SYNTAX_INVALID (BLOCKED)
Fallback candidate status: SYNTAX_VALIDATED (APPLIED)
Restoration: VERIFIED
Duration: 0.22 seconds
Progress: 4 / 4

================================================
FINAL COMPILER VALIDATION AND RECOVERY REPORT
================================================
Target: C Language AST and Parser Virtual Machine
Scenarios completed: 4 / 4
Passed: 4
Failed: 0
Automated Pytest Suite: 51 / 51 PASSED (100%)
Runtime: 0.76 seconds
```

Chapter-IV

CONCLUSION

The Intelligent Syntax Error Detection and Automatic Correction Using Compiler Techniques project successfully demonstrates an effective approach to identifying and correcting syntax errors in C programs. The system integrates lexical analysis, recursive-descent parsing, AST construction, and panic-mode error recovery to detect errors and minimize cascading diagnostics. By combining AI-based correction with deterministic rule-based repair and compiler-based validation, the system ensures that proposed corrections are checked before being applied.

This project highlights key compiler design concepts and demonstrates how they can be integrated with modern web technologies and automated code repair mechanisms. Features such as safe source editing, candidate ranking, and diagnostic comparison improve the reliability of the correction process and help prevent invalid modifications.

Overall, the project provides a practical implementation of compiler-based syntax analysis and automated error correction. It also offers a foundation for future enhancements, such as supporting a broader subset of the C language, incorporating semantic analysis, and extending correction capabilities to more complex programming errors.

Real-Life Applications:

- Used in **Code Editors and Integrated Development Environments (IDEs)** to identify syntax errors and assist programmers with corrections.
- Applied in **Automated Programming Assistance Tools** to generate and validate potential code repairs.
- Useful in **Compiler Design Education** for demonstrating lexical analysis, parsing, AST construction, and error recovery.
- Helps in **Software Development and Debugging** by identifying multiple syntax errors and reducing manual correction effort.
- Acts as an **Educational Tool for Programming Beginners** by providing structured diagnostics and visual comparisons of original and corrected code.

Chapter-V

REFERENCES

1. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D., *Compilers: Principles, Techniques, and Tools*, 2nd Edition, Pearson Education, 2006.
2. Cooper, K. D., & Torczon, L., *Engineering a Compiler*, 3rd Edition, Morgan Kaufmann, 2022.
3. ISO, *ISO/IEC 9899:2018 – Programming Languages – C*. https://www.iso.org/standard/74528.html
4. FastAPI Documentation, *FastAPI – Modern Python Web Framework*. https://fastapi.tiangolo.com/
5. Microsoft Documentation, *Monaco Editor Documentation*. https://microsoft.github.io/monaco-editor/
6. Google AI for Developers, *Gemini API Documentation*. https://ai.google.dev/gemini-api/docs
7. CrewAI Documentation, *Multi-Agent AI Framework*. https://docs.crewai.com/
8. Pytest Documentation, *Pytest – Python Testing Framework*. https://docs.pytest.org/